

Query Processing and Optimization

IN2401 – Database Management Systems (week 2)

Kanishka Nanayakkara

Lecturer

Department of Information Technology

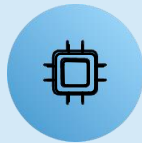
University Of Moratuwa

kanishkan@uom.lk



Learning Objectives

By the end of this lecture, you will be able to:



Identify the optimal way of executing a query



Design queries efficiently



What is Query Processing?

Query processing is the **series of steps** a DBMS follows to transform a high-level SQL query into a physical execution plan that retrieves the requested data.

01

Syntax Analysis

Parse and validate the SQL statement

02

Semantic Translation

Convert to internal representation

03

Optimization

Generate efficient execution strategy

04

Execution

Run the plan and return results

Example SQL Query

```
SELECT name  
FROM Student  
WHERE age > 20;
```

What the Database Must Do

1. Parse the SQL syntax and validate structure
2. Locate the Student table in storage
3. Retrieve relevant records efficiently
4. Filter rows where age > 20
5. Extract only the name attribute
6. Return results to the user

The Challenge

The database has multiple ways to execute this. Should it scan the entire table? Use an index? The optimizer must choose the **most efficient approach**.

What is Query Optimization?

Query optimization is an important component of any database system, focusing on finding the most efficient way to execute a relational query.

Relational Databases

Optimization is typically **built into the system**, automatically identifying the best execution plans for queries to ensure optimal performance.

Non-Relational Databases

Optimization often requires **manual effort**, where developers must actively design and structure queries for efficiency.



Objective of Query Optimizer

To improve efficiency and performance

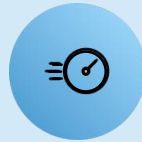
The primary goal is to find the most cost-effective execution plan for SQL queries, ensuring relational database systems operate with optimal speed and minimal resource usage.

Why Query Optimization Matters



Automatic Navigation

Relational database users simply state the required data, and the system intelligently handles the underlying process of locating that information.



Performance Achievement

Since many execution paths exist for a single query, the optimizer identifies and executes the most efficient, cost-effective plan.



Minimize Discrepancies

Given the significant speed gap between CPU and I/O, the optimizer prioritizes the cheapest operation set to minimize costly disk access.

Stages of Query Processing

Query processing occurs in three distinct stages, each with a specific responsibility:



Parsing and Translation

SQL query → Relational algebra expression



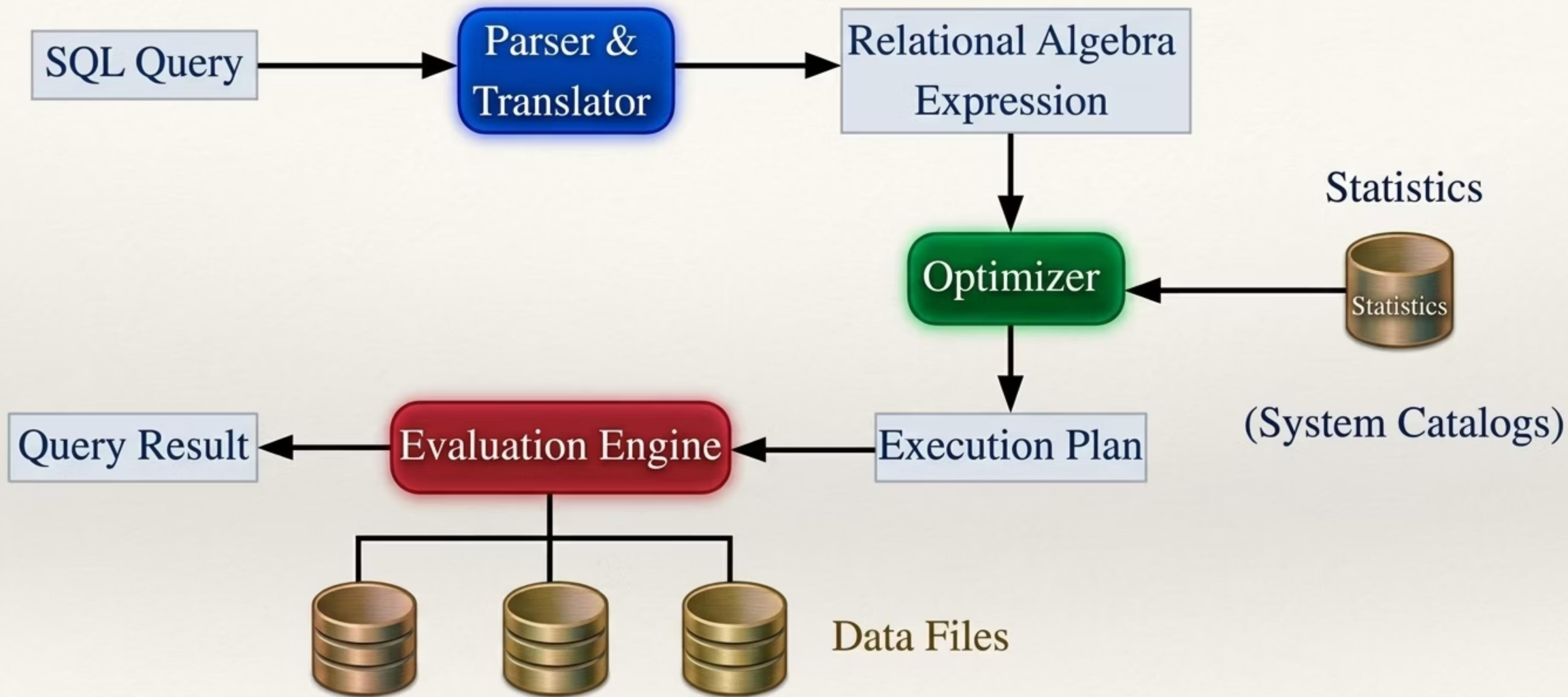
Query Optimization

Generate and cost multiple execution plans



Query Evaluation

Execute chosen plan and return results



Catalog Information

To estimate costs, the query optimizer relies on detailed statistics and metadata stored in the database catalog. This information helps in evaluating different execution plans.

Relations & Attributes

- **NR:** Number of tuples in the relation R.
- **BR:** Number of blocks that contain tuples of the relation R.
- **SR:** Size of a tuple of R.
- **FR:** Blocking factor; number of tuples from R that fit into one block. $FR = \lfloor NR/BR \rfloor$
- **V(A,R):** Number of distinct values for attribute A in R.
- **SC(A,R):** Selectivity of attribute A (average number of tuples satisfying an equality condition on A). $SC(A,R) = NR / V(A,R)$

Indexes

- **HTI:** Number of levels in index I (e.g., B+-tree).
- **LBI:** Number of blocks occupied by leaf nodes in index I (first-level blocks).
- **Vall:** Number of distinct values for the search key of index I.

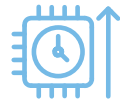
Cost of Query Execution

Database optimizers estimate the **total cost** of different execution plans using three main factors:



Disk I/O Cost

Number of disk blocks read or written. This is usually the **dominant factor** since disk access is slow compared to memory operations.



CPU Time

Processing required for comparisons, joins, sorting, and other operations. Important for complex queries with many calculations.



Memory Usage

Buffer space needed for intermediate results and sorting operations. Limited memory can force expensive disk-based operations.

📌 **Key insight:** Different execution plans for the same query can have vastly different costs. The optimizer's job is to find the lowest-cost plan.

Disk I/O Cost

Disk access is the predominant cost (in terms of time);

relatively easy to estimate;

**therefore,
number of block transfers
from/to disk is typically used as
measure.**

**– Simplifying assumption:
each block transfer has the
same cost.**

**Cost of algorithm (e.g., for join
or selection) depends on
database buffer size;**

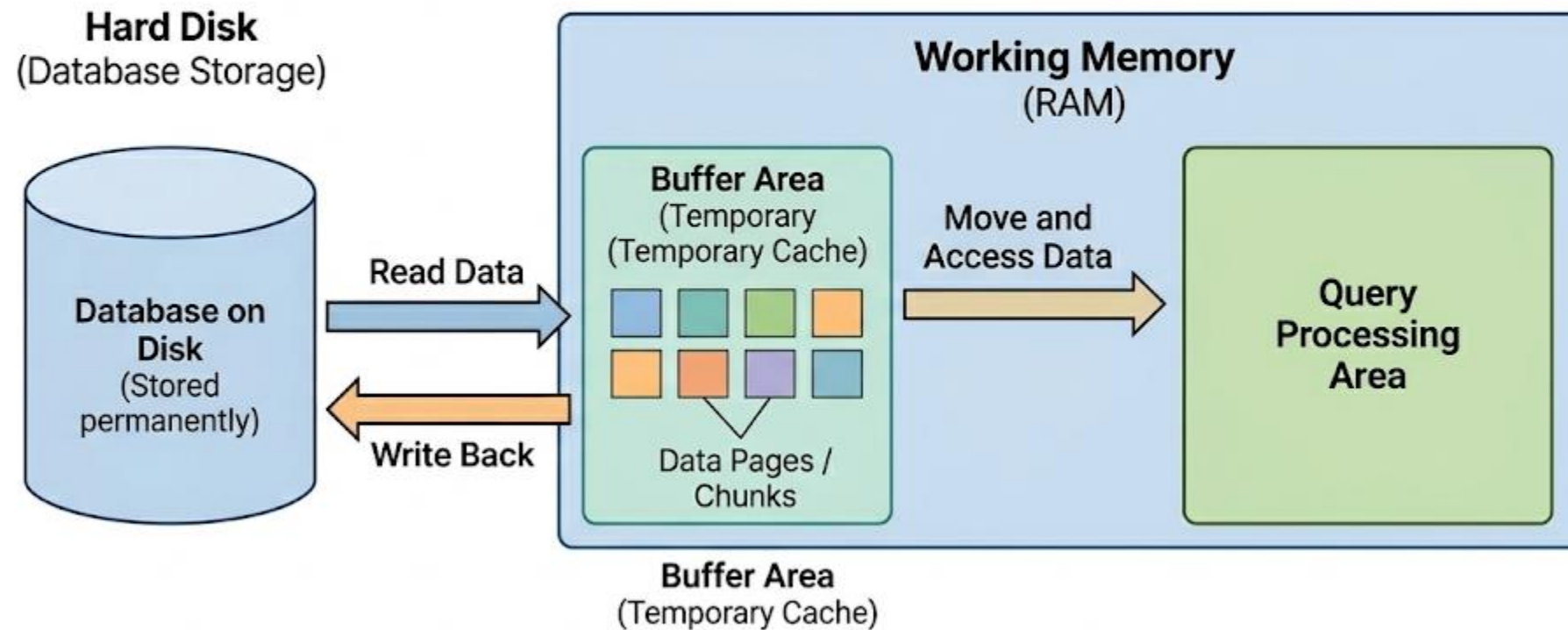
**more memory for DB buffer
reduces disk accesses.**

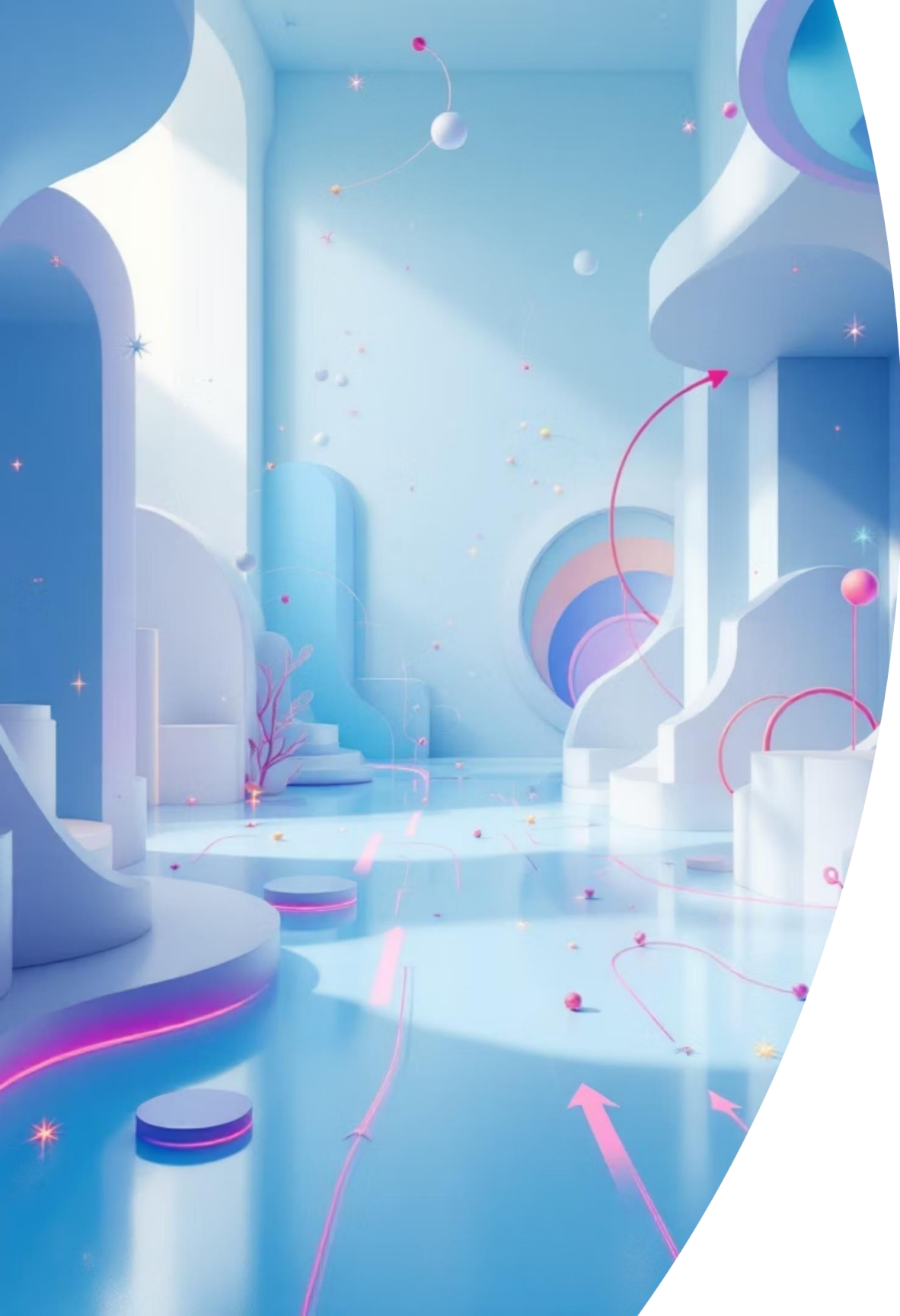
**Thus DB buffer size
is a parameter for estimating
cost.**

**We refer to the cost
estimate of algorithm S as
 $\text{cost}(S)$.**

**We do not consider cost
of writing output to disk.
(Cost of writing intermediary
results will be taken)**

DBMS Working Memory (RAM) Internal Data Handling





SQL to Relational Algebra Translation

Recap

Why Translation is Essential

SQL: The Human Interface

Structured Query Language provides an intuitive, English-like syntax that database administrators and developers use to interact with data. It abstracts away the complexity of data storage and retrieval mechanisms.

Relational Algebra: The Engine

Database Management Systems internally process queries using relational algebra operations. These mathematical foundations enable query optimization, execution planning, and efficient data manipulation at the system level.

Translation bridges the gap between human-readable commands and machine-executable operations, allowing the DBMS to analyze, optimize, and execute queries efficiently.



Relational Algebra Recap

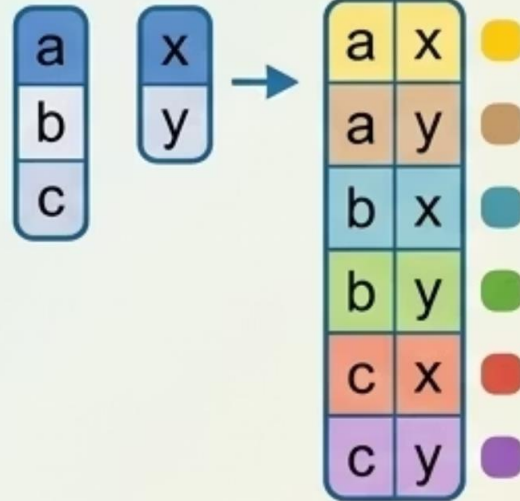
Restrict/Select



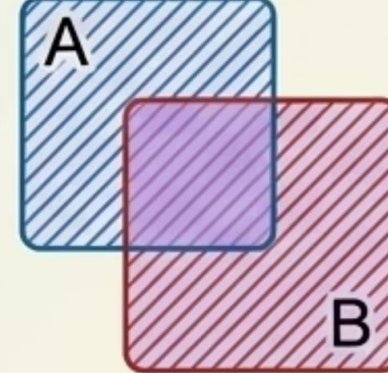
Project



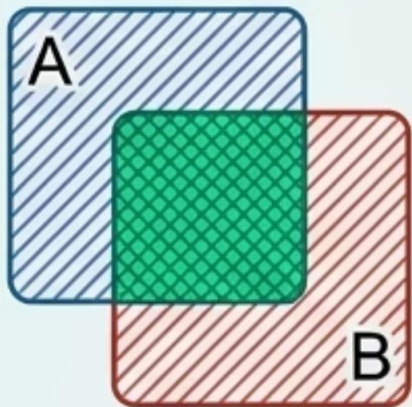
Product



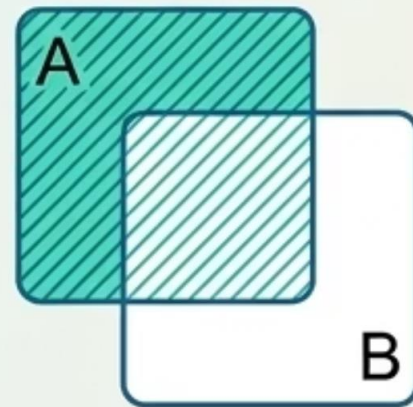
Union ($A \cup B$)



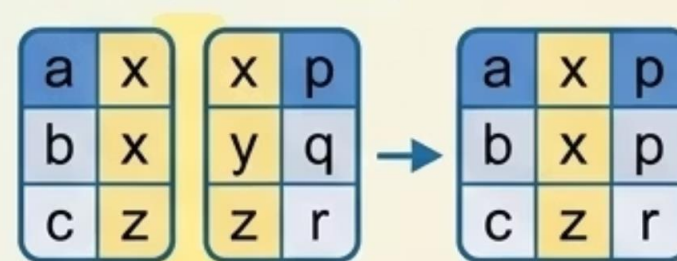
Intersect ($A \cap B$)



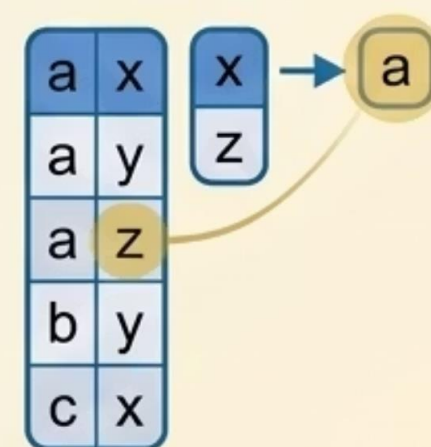
Difference ($A - B$)



Join (Natural)



Divide ($\div$)



	Name	Relational Algebra	SQL
Basic / Complete Set	select	$\sigma_{\text{age} > 18}(\text{patients})$	select * from patients where age > 18
	project	$\pi_{\text{patient_id, name}}(\text{patients})$	select patient_id, name from patients
	product	patients $\times$ medical_records	select * from patients, medical_records
	union (note: union tables must have the same number of columns and same data types)	$\pi_{\text{name}}(\text{patients}) \cup \pi_{\text{name}}(\text{doctors})$	select name from patients union select name from doctors
	difference (second table is not necessarily a subset of the first)	$\pi_{\text{name}}(\text{patients}) - \pi_{\text{name}}(\text{doctors})$	select name from patients minus select name from doctors
Derived	rename	$\rho_{\text{staff}}(\pi_{\text{patient_id, name}}(\text{patients}))$	select * from (select patient_id, name from patients) as staff
	intersection note: A intersect B = A-(A-B)	$\pi_{\text{name}}(\text{patients}) \cap \pi_{\text{name}}(\text{doctors})$	select name from patients intersect select name from doctors
	natural join note: lecture notes use star, but every other source seems to use bowtie	patients $\bowtie$ medical_records or patients $*$ medical_records	select * from patients natural join medical_records
	theta join note: you may sometimes see the '=' replaced with 'θ'	patients $\bowtie_{\text{patients.id=doctors.patient_id}}$ (doctors)	select * from patients join doctors on patients.id=doctors.patient_id



Query Optimization: The Next Step

Why Optimize?

Different plans can have dramatically different performance—sometimes seconds versus hours for the same query!

What's Next

Understanding how optimizers use statistics, indexes, and cost models to make intelligent decisions about query execution.

Example

Database Schema

- Student (StudNo, Sname, Gender, Address)
- Lending (LendingID, StudNo, BookNo)
- Book (BookNo, Title, Author, Edition)

Problem Query

Retrieve the names of students who have borrowed the book B1



Example

Database Schema

- Student (StudNo, Sname, Gender, Address)
- Lending (LendingID, StudNo, BookNo)
- Book (BookNo, Title, Author, Edition)

Problem Query

Retrieve the names of students who have borrowed the book B1

```
SELECT Distinct Student.Sname
FROM Student S, Lending L
WHERE S.StudNo = L.StudNo
AND L.BookNo = 'B1';
```

Option 1

$$\pi_{Sname} (\sigma_{L.BookNo='B1'} (Student \bowtie_{S.StudNo=L.StudNo} Lending))$$

Option 2

$$\pi_{Sname} (Student \bowtie_{S.StudNo=L.StudNo} (\sigma_{BookNo='B1'} (Lending)))$$

Assumptions

Key parameters for our analysis:

100

Students

Total number of student
records

10,000

Lending Records

Total number of borrowing
transactions

50

Book B1 Lendings

Number of times book B1
was borrowed

50

Memory Capacity

Tuples that can fit in
memory

Query Plan 1

A naive execution strategy without optimization:

01

Join Student and Lending

Combine all student records with
lending records

02

Select Book B1

Filter to only book B1 borrowings

03

Project Sname

Extract student names from results

- Student (StudNo, Sname, Gender, Address)
 - Lending (LendingID, StudNo, BookNo)
 - Book (BookNo, Title, Author, Edition)
- Database contains 100 students and 10,000 lendings.
 - Only 50 lendings are for book B1
 - Computers memory is sufficient to hold upto 50 tuples in the memory without having to write back to disk

Step 1 – Join Cost

The join operation generates a large intermediate result:

Join Characteristics

- For each Student row, every Lending row will be retrieved and tested
- Every lending row will match with one student row, so the number of joined rows in the intermediate relation is 10,000
- These have to be written back to the disk (Memory can keep only 50 tuples)

I/O Cost Calculation

$$\text{Cost} = (100 \times 10,000) + 10,000 = 1,010,000$$

Total: 1,010,000 I/O operations

Step 2 – Select Cost

- Database contains 100 students and 10,000 lendings.
- Only 50 lendings are for book B1
- Computers memory is sufficient to hold upto 50 tuples in the memory without having to write back to disk



Read Input

Reads 10,000 joined tuples in the results of Step 1 back in to the memory

Apply Filter

Match BookNo = 'B1'
Select produces a intermediate relation of only 50 matching tuples

Store Result

50 tuples fit in memory

I/O Cost: $10,000 + 0 = 10,000$

- Database contains 100 students and 10,000 lendings.
- Only 50 lendings are for book B1
- Computers memory is sufficient to hold upto 50 tuples in the memory without having to write back to disk

Step 3 – Project Cost

Final projection operates entirely in memory:

Input Size

50 tuples already loaded in memory

Operation

Extract Sname attribute from each tuple

I/O Cost

0 - No disk access required

I/O Cost: $0 + 0 = 0$

Total Cost: Plan 1

Cumulative I/O cost for the unoptimized execution:

$$1010000 + 10000 + 0$$

Join Cost

Step 1

Select Cost

Step 2

Project Cost

Step 3



Total I/O Cost: 1,020,000



Query Plan 2: The Optimized Approach

The same query can be executed more efficiently by changing the order of operations. Here's the optimized three-step process:

01	02	03
Selection First	Join with Students	Project Sname
Filter the Lending table to find books checked out by BookNo B1, reducing from 10,000 to 50 tuples	Combine the filtered results with the Student table to get student names	Extract only the student names as the final result

Step 1: Select First

- Database contains 100 students and 10,000 lendings.
- Only 50 lendings are for book B1
- Computers memory is sufficient to hold upto 50 tuples in the memory without having to write back to disk

The Filtering Operation

Start by reading all 10,000 tuples from the Lending table. Apply the selection condition to filter down to just 50 tuples where Book No equals B1.

These 50 tuples are stored in memory for the next step. The I/O cost for this selection operation is 10,000 page reads.

I/O Cost: 10,000 + 0 = 10,000

- Database contains 100 students and 10,000 lendings.
- Only 50 lendings are for book B1
- Computers memory is sufficient to hold upto 50 tuples in the memory without having to write back to disk

Step 2: Join Cost Analysis

Input Sizes

Join 100 students with 50 filtered lending tuples

Result Size

Produces 50 matching tuples after join

I/O Cost

Only 100 page reads required

By filtering first, the join operation becomes dramatically cheaper. Instead of processing 1,000,000 combinations, we now process just 5,000 potential matches (100×50).

I/O Cost: $100 + 0 = 100$

Step 3: Project Operation

- Reads a relation of 50 tuples which resulted in Step 2, which is already in the memory
- Final relation will have no more than 50 tuples, therefore can reside in the memory
- This is as same as in the Plan 1

- Database contains 100 students and 10,000 lendings.
- Only 50 lendings are for book B1
- Computers memory is sufficient to hold upto 50 tuples in the memory without having to write back to disk

I/O Cost: 0 + 0 = 0

Total Cost: Plan 2

10,000 + 100 + 0

Selection Cost

Page reads from Lending table

Join Cost

Page reads for join operation

Projection Cost

In-memory operation only

 **Total I/O Cost: 10,100**

Significantly lower than Plan 1's 1,020,000 operations

Performance Comparison

Plan 1 (Naive)

1,020,000 I/O operations

Cartesian product first, then filter

Plan 2 (Optimized)

10,100 I/O operations

Filter first, then join

Ratio

(Plan 1 to Plan 2) – $1,020,000 / 10,100$ – 102:1

The Key Insight

[Operation order matters tremendously](#). By selecting before joining, we achieve a 102x performance improvement—reducing execution time from potentially minutes to milliseconds.



Query Optimization Stages

Database systems automatically transform queries through multiple optimization stages:



Convert the query in to more suitable internal form

Convert SQL to internal form - • Query Tree • Relational Algebra



Canonical Form

Internal representation is further converted in to equivalent canonical way



Physical Operations

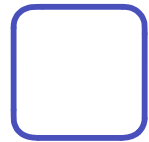
Transformed query is represented in a series of low-level operations, considering statistical information of the database



Plan Evaluation

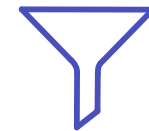
A set of candidate query plans are generated by combining a set of candidate implementation procedures

General Optimization Principles



Avoid Cartesian Products

Never compute full cross products—always filter or join with conditions first to reduce intermediate result sizes.



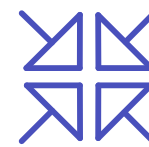
SELECT Before JOIN

Filter tables early to minimize the number of tuples participating in expensive join operations.



Use Indexes

Use available indexes to speed up selection and join operations without full table scans.



Reduce Intermediate Results

Apply projections and aggregations as early as possible to minimize memory and I/O overhead.



Query Trees



From Trees to Execution Plans

Multiple Strategies Exist

- Different join ordering possibilities
- Various access methods (index scan vs. full scan)
- Alternative operator implementations
- Different optimization heuristics

Query Optimizer's Role

The DBMS evaluates multiple equivalent query trees and selects the execution plan with the lowest estimated cost based on statistical information and system characteristics.



Key Insight: The same SQL query can be translated into different relational algebra expressions, each with distinct performance characteristics

Query Trees

Query trees visually represent relational algebra expressions, providing a structured approach to database operations and execution.

01

Leaf Nodes

Input relations (tables) form the base as leaf nodes.

03

Bottom-Up Execution

Processing begins from the lowest operations, applying to their operands.

05

Root Completion

Execution concludes when the root node generates the final result set.

02

Internal Nodes

Relational algebra operations define the internal nodes, connecting relations.

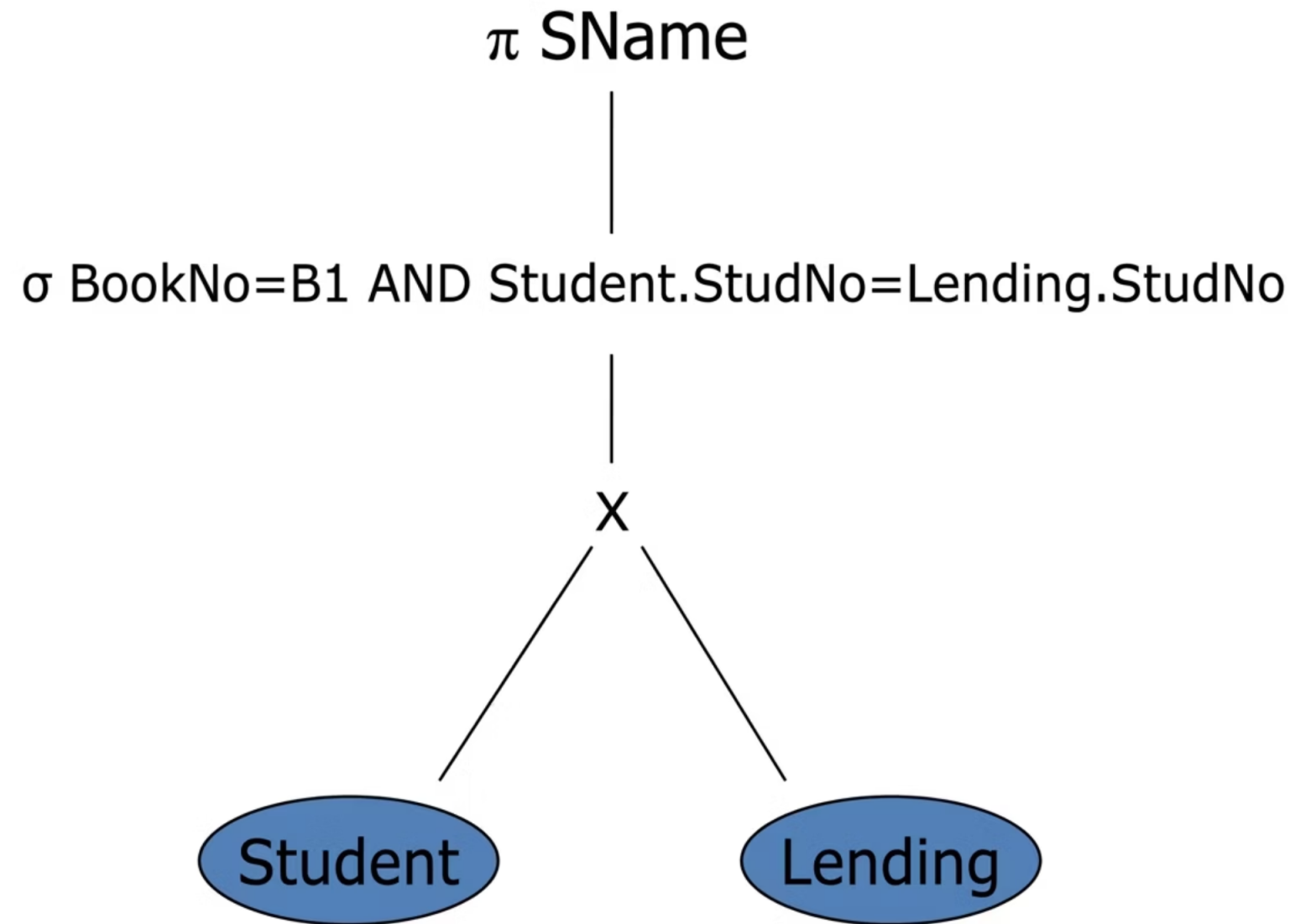
04

Resulting Relations

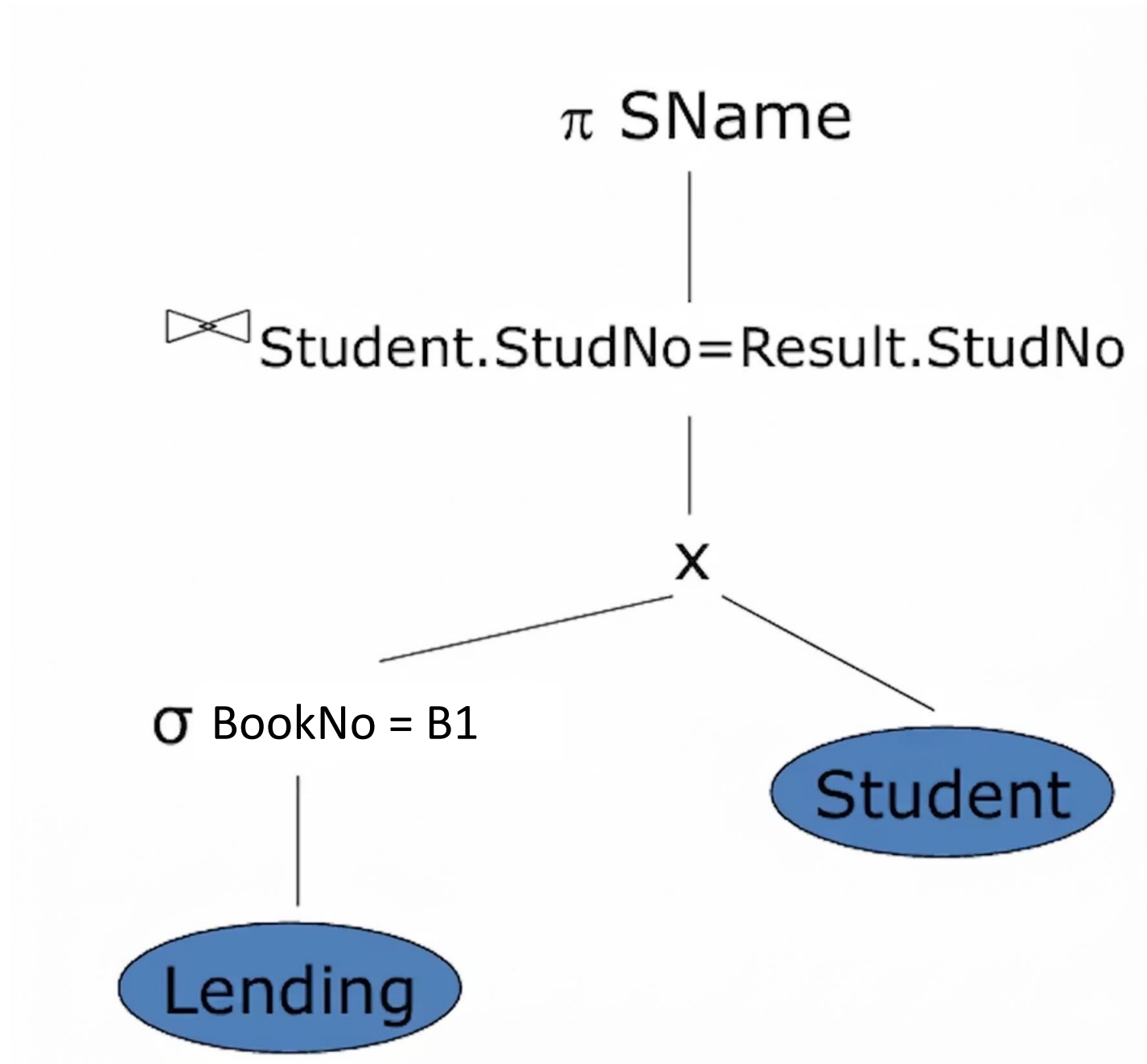
Each executed operation is replaced by its intermediate resulting relation.



Query Tree - plan 1



Query Tree - plan 2





Thank You